

Dokumentasi Tabungan Haji API

Bank Syariah Indonesia (BSI) · Materi Bootcamp

Dokumentasi Tabungan Haji API

REST API untuk simulasi **Tabungan Haji** Bank Syariah Indonesia (BSI) — materi bootcamp. API ini mengelola nasabah, rekening tabungan haji, transaksi setor/tarik, autentikasi JWT, estimasi tahun keberangkatan, serta export laporan bulanan ke CSV.

Daftar Dokumen

Dokumen	Isi
getting-started.md	Prasyarat, instalasi, konfigurasi <code>.env</code> , migrasi DB, menjalankan server
architecture.md	Struktur folder, pola berlapis (route/controller/service/schema), model data
api-reference.md	Referensi lengkap semua endpoint + contoh request/response + kode error
authentication.md	Alur login JWT, middleware <code>requireAuth</code> , logout & token blacklist
laporan-export.md	Script export laporan transaksi bulanan ke file CSV

Ringkasan Teknologi

Komponen	Pilihan
Runtime	Node.js + TypeScript (target ES2022, CommonJS)
Framework	Express 5
ORM / DB	Prisma 6 + PostgreSQL
Validasi	Zod
Keamanan	helmet, cors, JWT (<code>jsonwebtoken</code>), hashing <code>bcrypt</code>
Dev tooling	ts-node, nodemon

Fitur Utama

- **Manajemen nasabah** — registrasi (dengan password) + CRUD.
- **Buka rekening tabungan haji** — 1 nasabah = 1 rekening, nomor rekening unik anti-collision.
- **Setor & tarik** — atomik via transaksi DB, setor **idempotent** (header `Idempotency-Key`), minimum setor Rp100.000.
- **Lihat saldo & mutasi** — detail rekening dan riwayat mutasi (rekening koran).
- **Estimasi keberangkatan haji** — berdasarkan saldo (setoran awal porsi) & kuota tahunan.
- **Autentikasi JWT** — login, proteksi endpoint sensitif, logout dengan invalidasi token.
- **Export laporan bulanan** — script CLI menghasilkan file CSV.

Mulai Cepat

```
npm install
# Siapkan .env (lihat getting-started.md)
```

```
npx prisma migrate dev
npm run dev          # server di http://localhost:3000
```

Cek kesehatan: `GET http://localhost:3000/health` .

Catatan: dokumentasi ini ditulis untuk konteks pembelajaran/bootcamp. Beberapa keputusan (mis. blocklist token in-memory, belum ada otorisasi berbasis kepemilikan/role) disengaja demi kesederhanaan — lihat [authentication.md](#).

Getting Started

Panduan menyiapkan dan menjalankan Tabungan Haji API di lingkungan lokal.

Prasyarat

- **Node.js** 18+ (proyek diuji dengan tooling versi terbaru)
- **PostgreSQL** berjalan dan dapat diakses (default: `localhost:5432`)
- **npm**

1. Instalasi Dependency

```
npm install
```

Dependency runtime: `express` , `@prisma/client` , `prisma` , `zod` , `bcrypt` , `jsonwebtoken` , `helmet` , `cors` , `dotenv` .

Kendala jaringan korporat: di sebagian jaringan (mis. proxy/SSL inspection BSI), `npm install` paket baru bisa gagal dengan `UNABLE_TO_VERIFY_LEAF_SIGNATURE` . Untuk itu, type definition `@types/jsonwebtoken` & `@types/bcrypt` **belum** ter-install dan digantikan deklarasi lokal di `src/types/shims.d.ts` . Ganti dengan `@types/*` resmi saat jaringan memungkinkan. **Jangan** menonaktifkan verifikasi TLS sebagai jalan pintas.

2. Konfigurasi Environment (`.env`)

Buat file `.env` di root proyek:

```
PORT=3000
DATABASE_URL=postgresql://USER:PASSWORD@localhost:5432/tabungan_haji_online?schema=public
JWT_SECRET=ganti-dengan-string-acak-panjang
JWT_EXPIRES_SECONDS=3600
```

Variabel	Wajib	Keterangan
<code>PORT</code>	tidak	Port server, default <code>3000</code>
<code>DATABASE_URL</code>	ya	Connection string PostgreSQL
<code>JWT_SECRET</code>	ya*	Kunci penandatanganan JWT. *Ada fallback dev <code>dev-secret-change-me</code> , wajib diganti untuk selain dev
<code>JWT_EXPIRES_SECONDS</code>	tidak	Masa berlaku token (detik), default <code>3600</code> (1 jam)

Membuat secret acak:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

`.env` di-ignore git. Jangan commit kredensial.

3. Migrasi Database

```
npx prisma migrate dev
```

Perintah ini menerapkan migrasi di `prisma/migrations/` dan meng-generate Prisma Client ke `src/generated/prisma` (folder ini di-ignore git).

Migrasi yang ada:

- `..._init` — tabel `nasabah`, `tabungan_haji`, `transaksi`
- `..._add_nasabah_password` — kolom `password` pada `nasabah`

Cek status: `npx prisma migrate status`.

4. Menjalankan Server

```
npm run dev      # development (nodemon + ts-node, auto-reload)
```

atau build & jalankan hasil kompilasi:

```
npm run build    # output ke dist/  
npm start        # node dist/index.js
```

Server akan mencetak `Server berjalan di port 3000`.

5. Verifikasi

```
curl http://localhost:3000/health  
# {"status":"ok","service":"tabungan_haji_api","timestamp":"..."}
```

Script NPM

Script	Fungsi
<code>npm run dev</code>	Server mode dev (auto-reload)
<code>npm run build</code>	Kompilasi TypeScript ke <code>dist/</code>
<code>npm start</code>	Jalankan hasil build
<code>npm run export:laporan -- <YYYY-MM></code>	Export laporan transaksi bulanan ke CSV (lihat laporan-export.md)

Troubleshooting

Gejala	Penyebab & Solusi
<code>UNABLE_TO_VERIFY_LEAF_SIGNATURE</code> saat <code>npm install</code>	Proxy/SSL korporat. Lihat catatan di bagian Instalasi.
Property 'auth' does not exist on type 'Request' saat <code>ts-node</code>	Pastikan <code>tsconfig.json</code> memuat <code>"ts-node": { "files": true }</code> agar deklarasi ambient <code>src/types/*.d.ts</code> terbaca.
<code>Can't reach database server</code>	Pastikan PostgreSQL hidup & <code>DATABASE_URL</code> benar.
Error JWT <code>invalid signature</code>	<code>JWT_SECRET</code> berbeda antara saat token dibuat dan diverifikasi.

Arsitektur

Struktur Folder

```
TabunganHaji/
├─ docs/                # Dokumentasi (folder ini)
├─ prisma/
│   └─ schema.prisma    # Definisi model & datasource
│   └─ migrations/      # Riwayat migrasi SQL
├─ exports/             # Output CSV laporan (di-ignore git, dibuat saat export)
├─ src/
│   └─ index.ts          # Entry point: setup Express, middleware global, mount routes
│   └─ generated/prisma/ # Prisma Client hasil generate (di-ignore git)
│   └─ lib/
│       └─ prisma.ts     # Singleton PrismaClient (+ global omit password)
│       └─ jwt.ts        # Sign & verify JWT
│       └─ tokenBlocklist.ts # Blocklist token in-memory (logout)
│   └─ middleware/
│       └─ auth.middleware.ts # requireAuth (proteksi endpoint)
│   └─ types/
│       └─ express.d.ts  # Augmentasi Express.Request.auth
│       └─ shims.d.ts    # Deklarasi lokal jsonwebtoken & bcrypt
│   └─ scripts/
│       └─ exportLaporanBulanan.ts # Script export CSV bulanan
│   └─ modules/
│       └─ auth/          # login, logout
│       └─ nasabah/       # CRUD nasabah + registrasi
│       └─ tabungan/      # buka rekening, status, estimasi, dll
│       └─ transaksi/     # setor, tarik, mutasi, listing
├─ package.json
├─ tsconfig.json
└─ prisma.config.ts
```

Pola Berlapis (Layered) per Modul

Setiap domain di `src/modules/<domain>/` mengikuti pola yang sama:



Berkas	Tanggung jawab
*.route.ts	Mendaftarkan path HTTP, memasang middleware (mis. <code>requireAuth</code>)

Berkas	Tanggung jawab
*.controller.ts	Validasi input (Zod), memetakan error domain → kode HTTP, membentuk response
*.service.ts	Logika bisnis & akses database via Prisma
*.schema.ts	Skema validasi Zod + tipe input turunannya

Prinsip: controller tidak mengakses Prisma langsung; service tidak tahu soal HTTP. Service melempar `Error( 'KODE' )` (mis. `TABUNGAN_NOT_AKTIF`) yang dipetakan controller ke status HTTP.

Tech Stack & Keputusan Desain

- **Express 5** — error pada handler async otomatis diteruskan; saat ini belum ada error-hand

middleware kustom (error tak terduga → 500 default).

- **Prisma 6 + PostgreSQL** — Client di-generate ke `src/generated/prisma` .
- **BigInt untuk uang** — `saldo` & `nominal` bertipe `BigInt` agar bebas kehilangan presisi.

`src/index.ts` mem-patch `BigInt.prototype.toJSON` agar terserialisasi sebagai string di JSON.

- **Global omit password** — `src/lib/prisma.ts` mengatur `omit: { nasabah: { password: true } }`

sehingga password tidak pernah ikut terkirim, kecuali query login yang sengaja meng-override.

- **Zod** — validasi di boundary (controller), pesan error berbahasa Indonesia.

Model Data

```
erDiagram
    NASABAH ||--o{ TABUNGAN_HAJI : memiliki
    TABUNGAN_HAJI ||--o{ TRANSAKSI : mencatat

    NASABAH {
        string id PK
        string nik UK "16 digit"
        string nama
        string email UK
        string nomor_hp
        string password "nullable, bcrypt hash"
        datetime created_at
        datetime updated_at
    }
    TABUNGAN_HAJI {
        string id PK
        string nasabah_id FK
        string nomor_rekening UK
        bigint saldo "default 0"
        string status "AKTIF|TUTUP|SUSPEND"
        datetime dibuka_at
```



```

}
TRANSAKSI {
  string id PK
  string tabungan_id FK
  string jenis "SETOR|TARIK"
  bigint nominal
  bigint saldo_sebelum
  bigint saldo_sesudah
  string referensi UK "juga menyimpan Idempotency-Key"
  string metode "nullable, mis. QRIS"
  datetime waktu
}

```

Catatan field penting

- `Nasabah.password` — nullable; nasabah lama tanpa password tidak bisa login.
- `TabunganHaji.nomorRekening` — unik, di-generate 60 + 8 digit acak (retry bila bentrok).
- `TabunganHaji.status` — string (`AKTIF` default). Setor/tarik hanya untuk `AKTIF` .
- `Transaksi.referensi` — unik. Untuk setor idempotent, nilai `Idempotency-Key` disimpan

di kolom ini sehingga replay terdeteksi tanpa migrasi tambahan (lihat [api-reference.md](#)).

- `saldo_sebelum` / `saldo_sesudah` — snapshot saldo per transaksi → audit & rekening koran.

Konvensi Response Error

```
{ "error": "KODE_ERROR", "message": "pesan ramah pengguna" }
```

Untuk validasi:

```
{ "error": "VALIDATION_ERROR", "details": { "field": ["pesan"] } }
```

Daftar lengkap kode error ada di [api-reference.md](#).

API Reference

Base URL: `http://localhost:3000` Prefix API: `/api/v1`

- Body request & response berformat **JSON** (`Content-Type: application/json`).
- Endpoint terproteksi membutuhkan header `Authorization: Bearer <token>` (lihat [authentication.md](#)).
- Nilai uang (`saldo` , `nominal`) dikembalikan sebagai **string** (tipe BigInt).

Ringkasan Endpoint

Method	Path	Auth	Deskripsi
GET	<code>/health</code>	–	Health check
POST	<code>/api/v1/auth/login</code>	–	Login, dapatkan JWT
POST	<code>/api/v1/auth/logout</code>	✓	Logout (invalidasi token)
POST	<code>/api/v1/nasabah</code>	–	Registrasi nasabah
GET	<code>/api/v1/nasabah</code>	✓	Daftar nasabah
GET	<code>/api/v1/nasabah/:id</code>	✓	Detail nasabah (+ rekening)
PATCH	<code>/api/v1/nasabah/:id</code>	✓	Update nasabah
DELETE	<code>/api/v1/nasabah/:id</code>	✓	Hapus nasabah
POST	<code>/api/v1/tabungan-haji</code>	✓	Buka rekening
GET	<code>/api/v1/tabungan-haji</code>	✓	Daftar rekening
GET	<code>/api/v1/tabungan-haji/nasabah/:nasabahId</code>	✓	Rekening milik nasabah
GET	<code>/api/v1/tabungan-haji/:id</code>	✓	Detail rekening & saldo
GET	<code>/api/v1/tabungan-haji/:id/estimasi</code>	✓	Estimasi tahun keberangkatan
PATCH	<code>/api/v1/tabungan-haji/:id/status</code>	✓	Ubah status rekening
DELETE	<code>/api/v1/tabungan-haji/:id</code>	✓	Hapus rekening
POST	<code>/api/v1/tabungan-haji/:id/setor</code>	✓	Setor (idempotent)
POST	<code>/api/v1/tabungan-haji/:id/tarik</code>	✓	Tarik
GET	<code>/api/v1/tabungan-haji/:id/transaksi</code>	✓	Daftar transaksi rekening
GET	<code>/api/v1/tabungan-haji/:id/mutasi</code>	✓	Mutasi (rekening koran)
GET	<code>/api/v1/transaksi</code>	✓	Daftar transaksi (filter)
GET	<code>/api/v1/transaksi/:id</code>	✓	Detail transaksi

Health

GET /health

```
{ "status": "ok", "service": "tabungan_haji_api", "timestamp": "2026-05-28T04:45:27.448Z" }
```

Auth

Lihat detail di [authentication.md](#).

POST /api/v1/auth/login

Request:

```
{ "email": "siti.aminah@example.com", "password": "rahasia123" }
```

Response 200 :

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "nasabah": { "id": "...", "nik": "...", "nama": "Siti Aminah", "email": "...", "nomorHp": "..." }
}
```

Error: 400 validasi, 401 INVALID_CREDENTIALS .

POST /api/v1/auth/logout 

Header Authorization: Bearer <token> . Response 200 :

```
{ "message": "Logout berhasil, token dinonaktifkan" }
```

Nasabah

POST /api/v1/nasabah — Registrasi (publik)

Request:

```
{
  "nik": "3201010202910002",
  "nama": "Siti Aminah",
  "email": "siti.aminah@example.com",
  "nomorHp": "081298765432",
  "password": "rahasia123"
}
```

Aturan validasi:

Field	Aturan
nik	tepat 16 digit angka
nama	3–100 karakter
email	format email, ≤150 karakter, unik
nomorHp	pola 08xxxxxxxxxx (10–13 digit)
password	8–72 karakter

Response 201 (tanpa password):

```
{ "id": "...", "nik": "...", "nama": "Siti Aminah", "email": "...", "nomorHp": "...", "createdAt": "...",
  "updatedAt": "..." }
```

Error: 400 validasi, 409 DUPLICATE_ENTRY (nik/email sudah terdaftar).

GET /api/v1/nasabah 🔒

```
{ "data": [ { "id": "...", "nama": "..." } ], "total": 2 }
```

GET /api/v1/nasabah/:id 🔒

Mengembalikan nasabah beserta daftar tabungan. 404 NOT_FOUND bila tidak ada.

PATCH /api/v1/nasabah/:id 🔒

Body opsional (nama , email , nomorHp) — minimal satu field. Error: 400 , 404 NOT_FOUND , 409 DUPLICATE_ENTRY .

DELETE /api/v1/nasabah/:id 🔒

Response 204 . Error: 404 NOT_FOUND , 422 HAS_TABUNGAN (masih punya rekening).

Tabungan Haji

POST /api/v1/tabungan-haji — Buka rekening 🔒

Request:

```
{ "nasabahId": "3bba2c51-a629-4ba5-a70c-3227b7b8d8eb" }
```

Aturan bisnis: 1 nasabah = 1 rekening tabungan haji. Nomor rekening di-generate otomatis.

Response 201 :

```
{
  "id": "...", "nasabahId": "...", "nomorRekening": "6089344036",
```

```
"saldo": "0", "status": "AKTIF", "dibukaAt": "...",
"nasabah": { "id": "...", "nama": "Ahmad Fauzi", "...": "..." }
}
```

Kondisi	Status
Sukses	201
nasabahId tidak terdaftar	403 NASABAH_NOT_REGISTERED
Nasabah sudah punya rekening	409 DUPLICATE_TABUNGAN
Gagal membuat nomor unik (sangat jarang)	503 REKENING_GENERATION_FAILED

GET /api/v1/tabungan-haji 🔒

{ "data": [...], "total": n } — tiap item menyertakan nasabah .

GET /api/v1/tabungan-haji/nasabah/:nasabahId 🔒

Daftar rekening milik satu nasabah.

GET /api/v1/tabungan-haji/:id — Lihat saldo & detail 🔒

Response 200 (lihat contoh buka rekening). 404 NOT_FOUND bila rekening tidak ada.

GET /api/v1/tabungan-haji/:id/estimasi — Estimasi keberangkatan 🔒

Query (opsional):

Param	Default	Arti
kuotaTahunan	221000	Kuota haji nasional per tahun
nomorPorsi	1000000	Posisi antrian (nomor porsi)

Logika: butuh **setoran awal Rp25.000.000** untuk dapat porsi. Bila terpenuhi, $\text{tahunTunggu} = \text{ceil}(\text{nomorPorsi} / \text{kuotaTahunan})$ dan $\text{estimasiTahunBerangkat} = \text{tahunSekarang} + \text{tahunTunggu}$.

Contoh GET .../estimasi?kuotaTahunan=221000&nomorPorsi=1000000 saat saldo cukup → 200 :

```
{
  "rekening": { "id": "...", "nomorRekening": "...", "saldo": "25000000", "status": "AKTIF", "nasabah":
{"...": "..."} },
  "estimasi": {
    "setoranAwalMinimal": "25000000",
    "sudahMemenuhiSetoranAwal": true,
    "kekuranganSetoran": "0",
    "nomorPorsi": 1000000,
    "kuotaTahunan": 221000,
    "tahunSekarang": 2026,
    "tahunTunggu": 5,
    "estimasiTahunBerangkat": 2031
  }
}
```

```
}  
}
```

Bila saldo < 25 juta: `sudahMemenuhiSetoranAwal=false` , `kekuranganSetoran` terisi, `tahunTunggu` & `estimasiTahunBerangkat` bernilai `null` . `404 NOT_FOUND` bila rekening tidak ada.

Catatan: `kuotaTahunan` & `nomorPorsi` adalah asumsi yang dapat diatur, bukan integrasi data Kemenag.

PATCH `/api/v1/tabungan-haji/:id/status` 🔒

Request: { "status": "AKTIF" | "TUTUP" | "SUSPEND" } . Error: `400` , `404 NOT_FOUND` .

DELETE `/api/v1/tabungan-haji/:id` 🔒

Response `204` . Error: `404 NOT_FOUND` , `422 SALDO_TIDAK_NOL` , `422 HAS_TRANSAKSI` .

Transaksi (Setor / Tarik / Mutasi)

POST `/api/v1/tabungan-haji/:id/setor` — **Setor (idempotent)** 🔒

Header opsional: `Idempotency-Key: <8-50 karakter>` . Request:

```
{ "nominal": 250000, "metode": "QRIS" }
```

Aturan	Nilai
<code>nominal</code>	integer, minimum Rp100.000
<code>metode</code>	string opsional (<20), mis. <code>QRIS</code> , <code>TELLER</code>

Idempotency: bila `Idempotency-Key` dikirim ulang dengan request sama, transaksi **tidak** digandakan — server mengembalikan transaksi yang sama. Status `201` untuk transaksi baru, `200` untuk replay. Key sama dengan nominal/rekening berbeda → `409 IDEMPOTENCY_CONFLICT` .

Response `201` :

```
{  
  "id": "...", "tabunganId": "...", "jenis": "SETOR",  
  "nominal": "250000", "saldoSebelum": "0", "saldoSesudah": "250000",  
  "referensi": "STR1779943587042703", "metode": "QRIS", "waktu": "..."  
}
```

Kondisi	Status
Sukses (baru)	<code>201</code>
Replay idempotent	<code>200</code>
Nominal < 100rb / invalid	<code>400 VALIDATION_ERROR</code>

Kondisi	Status
Rekening tidak ada	404 NOT_FOUND
Rekening tidak AKTIF	422 TABUNGAN_NOT_AKTIF
Idempotency-Key dipakai untuk transaksi beda	409 IDEMPOTENCY_CONFLICT

POST /api/v1/tabungan-haji/:id/tarik — Tarik 🔒

Request: { "nominal": 100000, "metode": "TELLER" } (nominal integer positif).

Kondisi	Status
Sukses	201
Rekening tidak ada	404 NOT_FOUND
Rekening tidak AKTIF	422 TABUNGAN_NOT_AKTIF
Saldo tidak cukup	422 SALDO_TIDAK_CUKUP

GET /api/v1/tabungan-haji/:id/transaksi 🔒

{ "data": [...], "total": n } — daftar transaksi mentah,urut terbaru dulu.

GET /api/v1/tabungan-haji/:id/mutasi — Rekening koran 🔒

Response 200 :

```
{
  "rekening": { "id": "...", "nomorRekening": "...", "saldo": "650000", "status": "AKTIF",
    "nasabah": { "id": "...", "nik": "...", "nama": "..." } },
  "mutasi": [ { "jenis": "TARIK", "nominal": "100000", "saldoSebelum": "750000", "saldoSesudah": "650000",
    "...": "..." } ],
  "total": 3
}
```

Mutasi urut terbaru dulu, dengan saldo berjalan (saldoSebelum / saldoSesudah). 404 NOT_FOUND bila rekening tidak ada (dibedakan dari rekening kosong yang membalas total: 0).

GET /api/v1/transaksi 🔒

Query opsional: tabunganId , jenis (SETOR / TARIK). { "data": [...], "total": n } .

GET /api/v1/transaksi/:id 🔒

Detail satu transaksi (+ tabungan). 404 NOT_FOUND bila tidak ada.

Kode Error

HTTP	error	Makna
400	VALIDATION_ERROR	Input gagal validasi Zod (lihat details)
401	UNAUTHORIZED	Token tidak ada / tidak valid / kedaluwarsa
401	TOKEN_REVOKED	Token sudah di-logout (ada di blocklist)
401	INVALID_CREDENTIALS	Email atau password salah
403	NASABAH_NOT_REGISTERED	Buka rekening untuk nasabah yang belum terdaftar
404	NOT_FOUND	Resource tidak ditemukan
409	DUPLICATE_ENTRY	Nilai unik bentrok (nik/email/nomor rekening)
409	DUPLICATE_TABUNGAN	Nasabah sudah memiliki rekening tabungan haji
409	IDEMPOTENCY_CONFLICT	Idempotency-Key dipakai untuk transaksi berbeda
422	TABUNGAN_NOT_AKTIF	Operasi pada rekening non-AKTIF
422	SALDO_TIDAK_CUKUP	Saldo kurang untuk penarikan
422	SALDO_TIDAK_NOL	Hapus rekening yang masih bersaldo
422	HAS_TRANSAKSI	Hapus rekening yang punya riwayat transaksi
422	HAS_TABUNGAN	Hapus nasabah yang masih punya rekening
503	REKENING_GENERATION_FAILED	Gagal membuat nomor rekening unik setelah beberapa percobaan

Autentikasi & Otorisasi

API menggunakan **JWT (JSON Web Token)** stateless untuk autentikasi, dengan **blocklist in-memory** untuk mendukung logout (invalidasi token).

Komponen

Berkas	Peran
src/lib/jwt.ts	signAccessToken() & verifyAccessToken()
src/lib/tokenBlocklist.ts	Menyimpan jti token yang sudah logout (Map in-memory)
src/middleware/auth.middleware.ts	requireAuth — memverifikasi token & cek blocklist
src/types/express.d.ts	Menambahkan req.auth ke tipe Express Request
src/modules/auth/*	Endpoint login & logout

Alur

1. Registrasi

POST /api/v1/nasabah

(publik, password di-hash bcrypt)

2. Login

POST /api/v1/auth/login

→ terima { token, nasabah }

3. Akses

Authorization: Bearer <token>

pada endpoint terproteksi

4. Logout

POST /api/v1/auth/logout

→ jti token masuk blocklist

5. Token sama setelah logout

→ 401 TOKEN_REVOKED

Token

- Algoritma: HS256, ditandatangani dengan `JWT_SECRET`.
- Payload: `sub` (id nasabah), `email`, `jti` (UUID unik per token), `iat`, `exp`.
- Masa berlaku: `JWT_EXPIRES_SECONDS` (default 3600 detik / 1 jam).

Login

POST /api/v1/auth/login

{ "email": "siti.aminah@example.com", "password": "rahasia123" }

- Mengambil nasabah by email **termasuk** password (override global omit), lalu `bcrypt.compare`.
- Berhasil → `200 { token, nasabah }` (nasabah tanpa password).
- Gagal (email tidak ada / password salah / nasabah tanpa password) → `401 INVALID_CREDENTIALS`.

Middleware `requireAuth`

Dipasang di level router untuk endpoint sensitif:

```
// contoh: src/modules/tabungan/tabungan.route.ts
tabunganRoutes.use(requireAuth);
```

Perilaku:

1. Tidak ada header `Authorization: Bearer ...` → `401 UNAUTHORIZED`.
2. Token gagal verifikasi / kedaluwarsa → `401 UNAUTHORIZED`.
3. `jti` token ada di blocklist → `401 TOKEN_REVOKED`.
4. Valid → menempelkan `req.auth = { id, email, jti, exp }` lalu `next()`.

Cakupan proteksi

Grup	Status
<code>/api/v1/tabungan-haji/*</code>	Semua terproteksi
<code>/api/v1/transaksi/*</code>	Semua terproteksi
<code>/api/v1/nasabah</code> <code>POST</code> (registrasi)	Publik
<code>/api/v1/nasabah</code> <code>GET</code> / <code>PATCH</code> / <code>DELETE</code>	Terproteksi
<code>/api/v1/auth/login</code>	Publik
<code>/api/v1/auth/logout</code>	Terproteksi
<code>/health</code>	Publik

Logout & Invalidasi Token

`POST /api/v1/auth/logout` (butuh token):

- `requireAuth` mengisi `req.auth` (termasuk `jti` & `exp`).
- `jti` dimasukkan ke blocklist sampai waktu `exp`.
- Permintaan berikutnya dengan token yang sama → `401 TOKEN_REVOKED`.

`tokenBlocklist` membersihkan entri kedaluwarsa secara berkala (interval `unref`, tidak menahan proses).

Contoh (curl)

```
# Login
TOKEN=$(curl -s -X POST http://localhost:3000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"siti.aminah@example.com","password":"rahasia123"}' \
  | python -c "import sys,json;print(json.load(sys.stdin)['token'])")

# Akses terproteksi
curl http://localhost:3000/api/v1/tabungan-haji -H "Authorization: Bearer $TOKEN"
```

```
# Logout
curl -X POST http://localhost:3000/api/v1/auth/logout -H "Authorization: Bearer $TOKEN"

# Token sama → 401 TOKEN_REVOKED
curl http://localhost:3000/api/v1/tabungan-haji -H "Authorization: Bearer $TOKEN"
```

Batasan yang Disengaja (Konteks Bootcamp)

- **Blocklist in-memory** — sederhana, namun **hilang saat server restart** dan tidak berbagi

antar instance. Untuk produksi gunakan Redis atau tabel DB.

- **Belum ada otorisasi kepemilikan/role** — token valid mana pun bisa mengakses rekening

nasabah mana pun. Belum ada pengecekan `req.auth.id === tabungan.nasabahId` maupun peran admin/compliance. Tambahkan bila diperlukan.

- **JWT_SECRET fallback dev** — bila env tidak diset, dipakai `dev-secret-change-me` .

Wajib set secret kuat untuk selain pengembangan lokal.

Export Laporan Bulanan (CSV)

Export laporan transaksi per bulan diimplementasikan sebagai **script CLI** (bukan endpoint/modul), cocok untuk dijalankan admin/compliance secara berkala. Script menghasilkan file `.csv` di folder `exports/`.

- Script: `src/scripts/exportLaporanBulanan.ts`
- Output: `exports/laporan-transaksi-<YYYY-MM>.csv` (folder `exports/` di-ignore git)

Cara Menjalankan

```
npm run export:laporan -- 2026-05
```

atau langsung:

```
npx ts-node src/scripts/exportLaporanBulanan.ts 2026-05
```

Argumen `<YYYY-MM>` wajib dan harus valid (bulan `01 – 12`). Format salah → pesan penggunaan + exit code `1`.

Contoh keluaran terminal:

```
Laporan bulan 2026-05: 4 transaksi  
File CSV dibuat: .../TabunganHaji/exports/laporan-transaksi-2026-05.csv
```

Isi & Kolom CSV

Rentang waktu: seluruh bulan (`[YYYY-MM-01 00:00 UTC, bulan berikutnya)`),urut waktu menaik. Pemisah baris `\r\n`, nilai mengandung `,` `"` `\n` di-escape sesuai aturan CSV.

Kolom	Sumber
waktu	transaksi.waktu (ISO 8601)
referensi	transaksi.referensi
nomor_rekening	tabungan_haji.nomor_rekening
nik	nasabah.nik
nama	nasabah.nama
jenis	SETOR / TARIK
nominal	transaksi.nominal
saldo_sebelum	transaksi.saldo_sebelum
saldo_sesudah	transaksi.saldo_sesudah
metode	transaksi.metode (boleh kosong)

Contoh isi:

```
waktu,referensi,nomor_rekening,nik,nama,jenis,nominal,saldo_sebelum,saldo_sesudah,metode
2026-05-28T04:46:27.044Z,STR1779943587042703,6089344036,3201010101900001,Ahmad
Fauzi,SETOR,250000,0,250000,QRIS
2026-05-28T06:05:35.327Z,TRK1779948335325698,6089344036,3201010101900001,Ahmad
Fauzi,TARIK,100000,750000,650000,TELLER
```

Bila tidak ada transaksi pada bulan tersebut, file tetap dibuat berisi **baris header saja**.

Catatan

- Script terhubung langsung ke database (memakai singleton Prisma) lalu `prisma.$disconnect()` saat selesai.
- Karena berupa CLI, **tidak** melewati `requireAuth`; keamanannya bergantung pada akses ke server/DB.

Bila butuh export via HTTP dengan proteksi token, perlu menambahkan route khusus.

- Rentang bulan dihitung berbasis **UTC**.